Java Interview Notes

Method Overriding — Complete Study Guide

Method Overriding | Why Overriding | Dynamic Method Dispatch | Rules | Overloading vs Overriding

1. What is Method Overriding?

Method Overriding is a feature in Java where a child class provides its own implementation of a method that is already defined in its parent class. The method in the child class must have the same name, same parameters, and same return type as in the parent class.

Definition: Redefining a parent class method in the child class with the same signature to provide a specific implementation.

Key Characteristics

- Occurs between parent class and child class (requires inheritance).
- Method name, parameters, and return type must be identical.
- The child class version replaces the parent class version at runtime.
- **@Override** annotation is used to indicate that a method is being overridden.

Example — Animal and Dog

```
// Parent class
class Animal {
    void sound() {
        System.out.println("Animal makes a generic sound");
    }
}

// Child class — overrides sound()
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog says: Woof!");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat says: Meow!");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();   // Output: Dog says: Woof!

        Animal b = new Cat();
        b.sound();   // Output: Cat says: Meow!
    }
}
```

Note: Even though the reference type is Animal, the child class method is called at runtime. This is the power of Method Overriding.

2. Why Do We Use Method Overriding?

Reason 1 — Runtime Polymorphism

Method overriding is the foundation of runtime polymorphism. The same method call behaves differently depending on the actual object at runtime. This makes your code flexible and extensible.

Reason 2 — Code Reusability

The child class inherits all the existing code from the parent class. It only needs to override the methods that require different behavior. There is no need to rewrite everything from scratch.

Reason 3 — Providing Specific Behavior

A parent class method is generic by nature. Method overriding allows each child class to define its own specific behavior while still following the same method contract defined by the parent.

Reason 4 — Achieving Abstraction

You can define a general contract in the parent class (or interface) and let each subclass provide its own implementation. This is the basis of abstraction and interface-based programming.

Interview Answer: Method overriding is used to achieve runtime polymorphism, enable code reusability, allow child classes to provide specific implementations, and support abstraction.

3. Dynamic Method Dispatch

Definition: Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than at compile time. It is the way Java implements runtime polymorphism.

How It Works

When a parent class reference variable holds a child class object, calling an overridden method causes Java to determine which version of the method to execute based on the actual object type at runtime — not the reference type.

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s;                  // Parent reference

        s = new Circle();         // Object is Circle
        s.draw();                 // Output: Drawing a Circle

        s = new Rectangle();      // Object is Rectangle
        s.draw();                 // Output: Drawing a Rectangle
    }
}
```

Compile Time vs Runtime

	Compile Time	Runtime (JVM)

Reference type known?	Yes — Shape	Yes — Shape
Object type known?	No	Yes — Circle or Rectangle
Which method to call?	Not decided yet	Decided here
Who decides?	Compiler	JVM

Key Point: Dynamic Method Dispatch and Runtime Polymorphism are the same concept — just different names used in different contexts.

4. Rules of Method Overriding

All of the following rules must be satisfied for a valid override. Violating any one of them will cause a compile-time error.

Rule 1 — Same Method Name

- The method name in the child class must be exactly the same as in the parent class.

Rule 2 — Same Parameter List

- The number, type, and order of parameters must be identical to the parent class method.
- If parameters are different, it becomes method overloading — not overriding.

Rule 3 — Same or Covariant Return Type

- The return type must be the same as the parent method.
- A covariant return type is also allowed — the child's return type can be a subclass of the parent's return type.

```
class Animal {
    Animal getInstance() { return new Animal(); }
}

class Dog extends Animal {
    @Override
    Dog getInstance() { return new Dog(); }  // Covariant return type — valid!
}
```

Rule 4 — Access Modifier Cannot Be More Restrictive

- The child class method's access level can be the same or broader — but never narrower.

Parent Access	Child Access	Valid?
public	public	YES
public	protected	NO — more restrictive
protected	public	YES — broader access
protected	protected	YES
private	any	NO — private methods are not inherited or overridden

Rule 5 — static Methods Cannot Be Overridden

- Static methods belong to the class, not to an object. They cannot be overridden.
- If you define the same static method in a child class, it is called Method Hiding — not overriding.

```
class Parent {
    static void show() { System.out.println("Parent static"); }
}
class Child extends Parent {
    static void show() { System.out.println("Child static"); }  // Method Hiding
}

// Parent.show() → "Parent static"
// Child.show()  → "Child static"   (resolved at compile time, not runtime)
```

Rule 6 — final Methods Cannot Be Overridden

- A method declared as final in the parent class cannot be overridden in any child class.
- Attempting to override a final method results in a compile-time error.

```
class Parent {
    final void display() {
        System.out.println("This method cannot be overridden.");
    }
}

class Child extends Parent {
    void display() { }  // COMPILE ERROR: cannot override final method
}
```

Rule 7 — Use @Override Annotation

- Writing @Override is not mandatory, but it is strongly recommended as a best practice.

- It instructs the compiler to verify that the method is actually overriding a parent method.
- If you accidentally misspell the method name, the compiler will immediately flag an error.

Memory Tip: Rules summary — Same Name, Same Parameters, Same Return Type, Access cannot narrow, Not static, Not final, Use @Override.

5. Method Overloading vs Method Overriding

Point	Method Overloading	Method Overriding
Where it occurs	Within the same class	Between parent and child class
Method name	Must be the same	Must be the same
Parameters	Must be different	Must be identical
Return type	Can be different	Must be same (or covariant)
Inheritance required?	No	Yes
Resolved at	Compile time	Runtime (by JVM)
Type of polymorphism	Compile-time polymorphism	Runtime polymorphism
@Override annotation	Not applicable	Should be used
Works with static?	Yes	No
Works with private?	Yes	No
Works with final?	Yes	No

Code Comparison

```
// ──────────── METHOD OVERLOADING ────────────
class Calculator {
    int add(int a, int b) { return a + b; }             // 2 int params
    int add(int a, int b, int c) { return a + b + c; }  // 3 int params
    double add(double a, double b) { return a + b; }    // double params
    // Same name, different parameters — resolved at compile time
}

// ──────────── METHOD OVERRIDING ────────────
class Animal {
    void sound() { System.out.println("Generic sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Woof!"); }
```

```
    // Same name, same parameters — resolved at runtime
}
```

One-Line Difference: Overloading = same class, different parameters, compile time. Overriding = child class, same parameters, runtime.

Quick Revision — One Line Each

Topic	One-Line Summary
Method Overriding	Redefining a parent class method in the child class with the same signature.
Why Overriding	Runtime polymorphism, code reuse, specific behavior, abstraction.
Dynamic Method Dispatch	JVM decides at runtime which overridden method to call based on the actual object.
@Override	Tells the compiler to verify the override — catches typos and invalid overrides.
Covariant Return Type	Child method can return a subtype of the parent method's return type.
Method Hiding	Redefining a static method in a child class — not overriding, resolved at compile time.
Overloading	Same class, same method name, different parameters — compile-time polymorphism.
Overriding	Parent-child, same method name, same parameters — runtime polymorphism.

All the best for your Java interview!